

Laboratory of Decision Support Systems

Barbieri Bruno

Braghin Federica

Vannucci Nicholas

December 2025

Abstract

This is a concise abstract summarizing the motivation, method, and main results of the work.

1 Introduction

Introduce the problem, context, and contributions.

2 Related Work

Brief overview of relevant literature.

3 Methodology

Describe the proposed method or theoretical framework.

3.1 Assignment 4 – Data Warehouse Schema Design

L'Assignment 4 richiedeva il passaggio da un database operativo a un data warehouse orientato all'analisi, con l'obiettivo di progettare uno schema multidimensionale in grado di analizzare il numero di stream registrati a un mese dalla pubblicazione di una canzone (*Streams1Month*).

Lo schema fornito nella consegna era indicato come riferimento non vincolante, lasciando libertà progettuale nella definizione della granularità, delle dimensioni e delle relazioni. Seguendo l'approccio discusso a lezione, la progettazione è partita dall'analisi delle *business questions* e delle query che si intendevano supportare.

Da tale analisi è emerso che la misura centrale del data warehouse è unica:

- **Streams1Month**, ovvero il numero di stream registrati a un mese dalla pubblicazione del brano.

A partire da questa misura sono state individuate le dimensioni rilevanti per l'analisi:

- **DimSong**, che rappresenta la canzone pubblicata;
- **DimArtist**, che rappresenta gli artisti che hanno partecipato alla canzone;
- **DimDate**, che rappresenta la data di pubblicazione del brano;
- **DimArtistGeography**, che rappresenta il luogo di nascita dell'artista;
- **DimLyrics**, che rappresenta il testo del brano e le sue componenti;

- **DimSymphony**, che rappresenta le caratteristiche acustiche del brano;
- **DimAlbum**, che rappresenta il contesto editoriale della canzone, non necessariamente coincidente con un album tradizionale.

Un'ulteriore scelta progettuale rilevante è stata quella di non utilizzare le *natural key* presenti nei dataset originali, in quanto fortemente duplicate e poco affidabili. Sono state pertanto introdotte chiavi surrogate univoche, generate tramite UUID, al fine di garantire coerenza, stabilità e corretta identificazione delle dimensioni.

Evoluzione dello schema e scelte progettuali

In una prima fase progettuale è stata considerata una fact table denominata *PublishedSong*, con granularità definita come *una riga per canzone pubblicata da un artista*, collegata alle dimensioni Song, Artist, Date, Lyrics e Symphony.

In tale schema, la gestione degli artisti in featuring sarebbe stata possibile esclusivamente tramite l'introduzione di una *bridge table* tra *DimSong* e *DimArtist*. Tuttavia, questo approccio presentava alcune criticità rilevanti. In primo luogo, l'uso della bridge table avrebbe condotto a una struttura che non aderiva pienamente né allo *star schema* né allo *snowflake schema*, aumentando il numero di join necessari per l'esecuzione delle query analitiche e riducendo l'efficienza complessiva del sistema.

In secondo luogo, la gestione dei featuring risultava meno naturale dal punto di vista analitico, poiché il ruolo dell'artista (artista primario o featuring) non era direttamente associato alla misura analizzata, ma derivato da una struttura intermedia.

Sono state inoltre considerate altre alternative progettuali, tra cui:

- l'introduzione di un attributo *Role* nella bridge table;
- l'utilizzo di una *degenerate dimension* per distinguere tra artisti primari e featuring;
- la creazione di una dimensione di gruppo di artisti tra la fact table e la dimensione Artist.

Sebbene concettualmente corrette, tali soluzioni introducevano una maggiore complessità strutturale o rendevano meno immediate le analisi per artista, risultando meno adatte agli obiettivi dell'assignment.

Schema finale del Data Warehouse

La soluzione finale adottata è stata la creazione di una fact table denominata *FactParticipation*, che modella esplicitamente la partecipazione di un artista a una canzone.

La granularità finale è definita come:

una riga = partecipazione di un artista a una canzone in una determinata data.

Di conseguenza, la stessa canzone può comparire più volte all'interno della fact table, una volta per ciascun artista coinvolto. Questo comportamento è atteso ed è il risultato di una granularità più fine, scelta in modo consapevole per rappresentare correttamente la relazione molti-a-molti tra artisti e canzoni, evitando duplicazioni semantiche delle misure.

Il ruolo dell'artista è rappresentato tramite l'attributo *IsPrimary*, modellato come *degenerate dimension* all'interno della fact table:

- *IsPrimary* = 1 identifica l'artista principale;
- *IsPrimary* = 0 identifica gli artisti in featuring.

La fact table è collegata direttamente alle dimensioni *DimSong*, *DimArtist* e *DimDate*, mentre le dimensioni Song e Artist generano strutture a *snowflake schema* rispettivamente verso *DimAlbum*, *DimLyrics*, *DimSymphony* e *DimArtistGeography*. L'insieme delle foreign key identifica univocamente i record della fact table, in accordo con quanto discusso a lezione.

La dimensione *DimDate* è stata collegata direttamente alla fact table, in quanto rappresenta il riferimento temporale dell'evento analizzato (stream a un mese dalla pubblicazione). Altre date semanticamente differenti, come la data di rilascio dell'album, sono state mantenute come attributi all'interno delle rispettive dimensioni, evitando ambiguità e join aggiuntivi.

Implementazione

Una volta definito lo schema multidimensionale del Data Warehouse, è stata realizzata anche la sua implementazione fisica sul database server utilizzando SQL Server Management Studio. In particolare, è stato scritto ed eseguito uno script SQL per la creazione delle tabelle dimensionali e della fact table, in piena coerenza con lo schema logico descritto.

Le tabelle sono state inizialmente create vuote, definendo esplicitamente:

- gli attributi e i relativi tipi di dato;
- le chiavi primarie;
- le chiavi esterne che implementano le relazioni tra fact table e dimensioni.

Le foreign key riflettono correttamente la struttura del Data Warehouse, garantendo allineamento tra modello concettuale, logico e implementazione fisica. Sono stati inoltre introdotti vincoli e scelte sui tipi di dato per garantire coerenza e qualità dei dati. In particolare:

- l'attributo *Gender* della dimensione *DimArtist* è stato vincolato ai soli valori ammessi (*M* o *F*) tramite una *check constraint*;
- i tipi di dato numerici sono stati scelti in base alla natura degli attributi, utilizzando *INT* per conteggi discreti, *FLOAT* per misure continue e *BIT* per attributi booleani come *IsPrimary*;
- la misura *Streams1Month* è stata definita come attributo numerico adeguato a rappresentare valori di conteggio di grandi dimensioni.

Questa fase ha permesso di separare in modo chiaro la progettazione dello schema dalla fase di popolamento dei dati, che viene affrontata negli assignment successivi. Il caricamento effettivo dei dati nel Data Warehouse è infatti realizzato nell'Assignment 6, tramite strumenti ETL, a partire dai file CSV prodotti nell'Assignment 5.

3.2 Assignment 5 – Data Preparation and Dataset Splitting

L'Assignment 5 richiedeva lo sviluppo di un programma Python in grado di suddividere i dati di input in file distinti, uno per ciascuna dimensione e per la fact table del Data Warehouse progettato nell'Assignment 4, al fine di preparare i dataset per le successive fasi di caricamento.

Per soddisfare tale richiesta, il processo di *data preparation* è stato articolato in due fasi consecutive e complementari:

- pulizia e normalizzazione dei dati;
- suddivisione ed esportazione dei dataset, uno per ciascuna tabella del Data Warehouse.

Pulizia e normalizzazione dei dati

La prima fase è stata implementata tramite lo script `prepare_dw_datasets.py`, il cui obiettivo è rendere i dati coerenti con lo schema multidimensionale progettato.

Lo script carica i dataset in formato JSON dalla directory `dataset/correct_ids` ed esegue una serie di operazioni di pulizia e normalizzazione prima dello *split* vero e proprio. In particolare, vengono effettuate le seguenti operazioni:

- uniformazione dei tipi di dato (interi, float e stringhe);
- gestione consistente dei valori mancanti e non validi;
- normalizzazione degli attributi testuali;
- correzione di incongruenze sintattiche ereditate dai dataset originali;
- generazione e validazione delle chiavi surrogate, in linea con lo schema del Data Warehouse;
- normalizzazione della dimensione temporale e derivazione dell'attributo *Season*.

Il risultato di questa fase è un insieme di dataset puliti e coerenti, salvati nella directory `dataset/cleaned_json`, che rappresentano una versione intermedia pronta per essere suddivisa secondo le tabelle del Data Warehouse.

Esportazione

La seconda fase è stata implementata tramite lo script `export_dw_csv.py`, che realizza esplicitamente quanto richiesto dalla consegna, ovvero la suddivisione dei dati in file separati, uno per ciascuna tabella del Data Warehouse.

A partire dai dataset normalizzati, lo script genera un file CSV per ogni dimensione e per la fact table. Ogni file CSV contiene esclusivamente gli attributi previsti dallo schema del Data Warehouse ed è costruito in modo da rispettarne la granularità.

In particolare, la fact table *FactParticipation* mantiene la granularità definita nell'Assignment 4, ovvero:

una riga per partecipazione artista–canzone.

Essa include la misura *Streams1Month* e l'attributo *IsPrimary*, modellato come *degenerate dimension*.

I file CSV prodotti costituiscono quindi l'output finale dell'Assignment 5 e rappresentano l'input diretto per le successive fasi di caricamento del Data Warehouse.

L'Assignment 5 ha permesso di trasformare i dati di partenza in un insieme di file strutturati, uno per ciascuna tabella del Data Warehouse, come richiesto dalla consegna. La separazione tra fase di normalizzazione e fase di esportazione consente di ottenere dati puliti, coerenti e direttamente utilizzabili, facilitando le successive attività di popolamento e analisi.

3.3 Assignment 6 – Data Uploading with Python

L'Assignment 6 richiedeva lo sviluppo di un programma Python in grado di popolare il database del Data Warehouse (*Group_ID_8_DB*), rispettando lo schema e le relazioni definite nell'Assignment 4 e utilizzando i dati preparati negli assignment precedenti.

Poiché il caricamento completo dei dati può risultare oneroso in termini di tempo e risorse, è stato richiesto di progettare il codice in modo appropriato, tenendo conto sia delle prestazioni sia della stabilità del processo.

Il caricamento dei dati è stato implementato tramite uno script Python che utilizza la libreria `pyodbc` per stabilire una connessione a *SQL Server*. La connessione viene effettuata specificando

server, database e credenziali, consentendo l'esecuzione diretta di istruzioni SQL dal codice Python. Una volta stabilita la connessione, viene creato un cursore che permette di eseguire operazioni di inserimento sulle tabelle del Data Warehouse.

Per garantire il rispetto dei vincoli di integrità referenziale, le tabelle vengono caricate seguendo un ordine prestabilito, coerente con le relazioni di *foreign key* definite nello schema:

- dimensioni indipendenti;
- dimensioni che dipendono da altre dimensioni;
- fact table.

In particolare, le dimensioni vengono caricate prima della fact table *FactParticipation*, assicurando che tutte le chiavi esterne siano già presenti nel database al momento dell'inserimento dei fatti.

Prima di avviare il caricamento, lo script verifica se una tabella è già popolata tramite una semplice query di conteggio. Se una tabella risulta non vuota, il caricamento viene saltato, evitando duplicazioni accidentali dei dati. Questo controllo rende lo script riutilizzabile e consente di rilanciare il processo senza dover ripulire manualmente il database.

I dati vengono letti dai file CSV prodotti nell'Assignment 5. Per ogni tabella, lo script:

- legge l'intestazione del file per determinare le colonne;
- costruisce dinamicamente la query di inserimento;
- effettua una conversione esplicita dei tipi di dato per alcuni attributi critici, in particolare:
 - *Streams1Month*, convertito in valore numerico;
 - *IsPrimary*, convertito in valore booleano/integer;
- gestisce correttamente i valori mancanti, convertendoli in NULL.

Poiché il caricamento completo dei dati può richiedere tempi significativi, l'inserimento delle tuple è stato progettato in modalità *batch*. I record vengono accumulati in gruppi di dimensione fissa e inseriti nel database tramite operazioni di inserimento multiplo, seguite da un commit esplicito.

Questa scelta consente di ridurre il numero di comunicazioni tra Python e il database, migliorare le prestazioni complessive del caricamento e rendere il processo più stabile in presenza di grandi volumi di dati.

Lo script presentato rappresenta la versione finale del processo di caricamento: durante lo sviluppo sono stati effettuati diversi test e lanci intermedi per verificare la connessione al database e il corretto inserimento dei dati, prima di consolidare la soluzione a batch adottata.

L'Assignment 6 conclude quindi il processo di costruzione del Data Warehouse, collegando la fase di preparazione dei dati alla loro effettiva persistenza nel database in modo automatico, coerente e ripetibile.

4 Experiments

Explain experimental setup, data, and evaluation.

5 Results

Present and discuss results.

6 Conclusion

Summarize findings and outline future work.